Informe - Etapa 4

Autor: Nicolás Cid
Materia: Compiladores e Intérpretes

Diagrama de Clases de los AST

Este diagrama es una versión simplificada (no exhaustiva) del diagrama de clases, pensada para facilitar la comprensión de la jerarquía de nodos de los AST. Solo se incluye la información relevante al chequeo de sentencias: se omitieron los constructores y los métodos auxiliares privados, y de la tabla de símbolos solo aparecen `Unidad` y `Atributo`, que son las entidades que guardan un AST (el bloque de un método o constructor y el inicializador de un atributo). También se omitieron las flechas de dependencia, agregación, composición y asociación, que dificultan la lectura del gráfico: las relaciones entre clases pueden deducirse a partir del tipo de los atributos de cada clase.

Notación de visibilidad utilizada:

- `+`: público.
- `-`: privado.
- `#`: protegido.
- `~`: visible solo dentro del paquete.

El ícono junto al nombre de cada clase indica su tipo:

- `C`: clase concreta.
- `A`: clase abstracta. Sus métodos abstractos se muestran en cursiva.

Para que entre en el ancho de la página, el diagrama se divide en tres partes: las sentencias (junto con `Unidad` y `Atributo`), las expresiones con operadores, literales y accesos, y las expresiones con llamadas, creaciones y encadenados. En la tercera, `NodoExpresion` aparece sin miembros porque ya se detalla en la segunda.

Diagrama de clases de los AST: sentencias

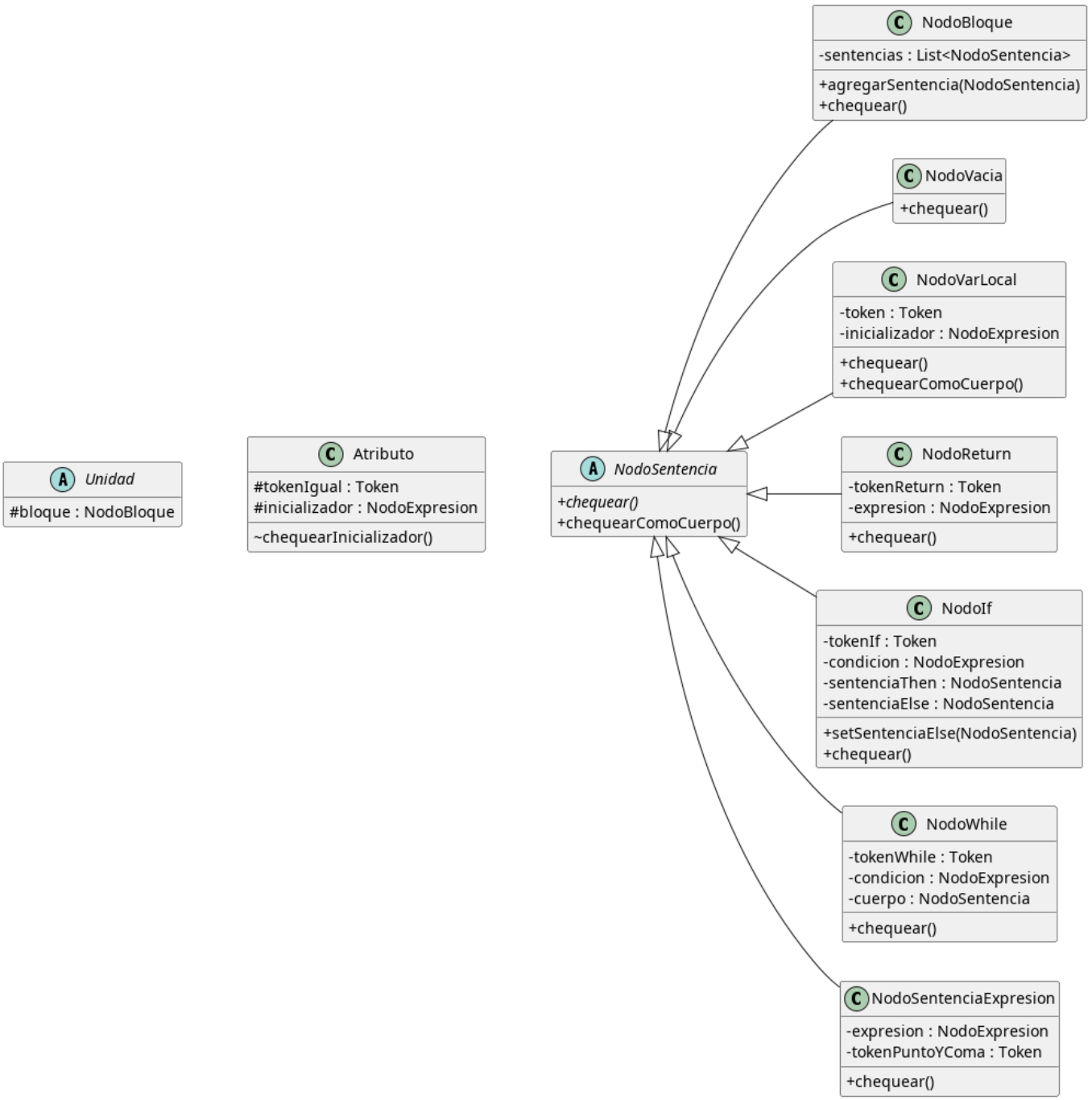
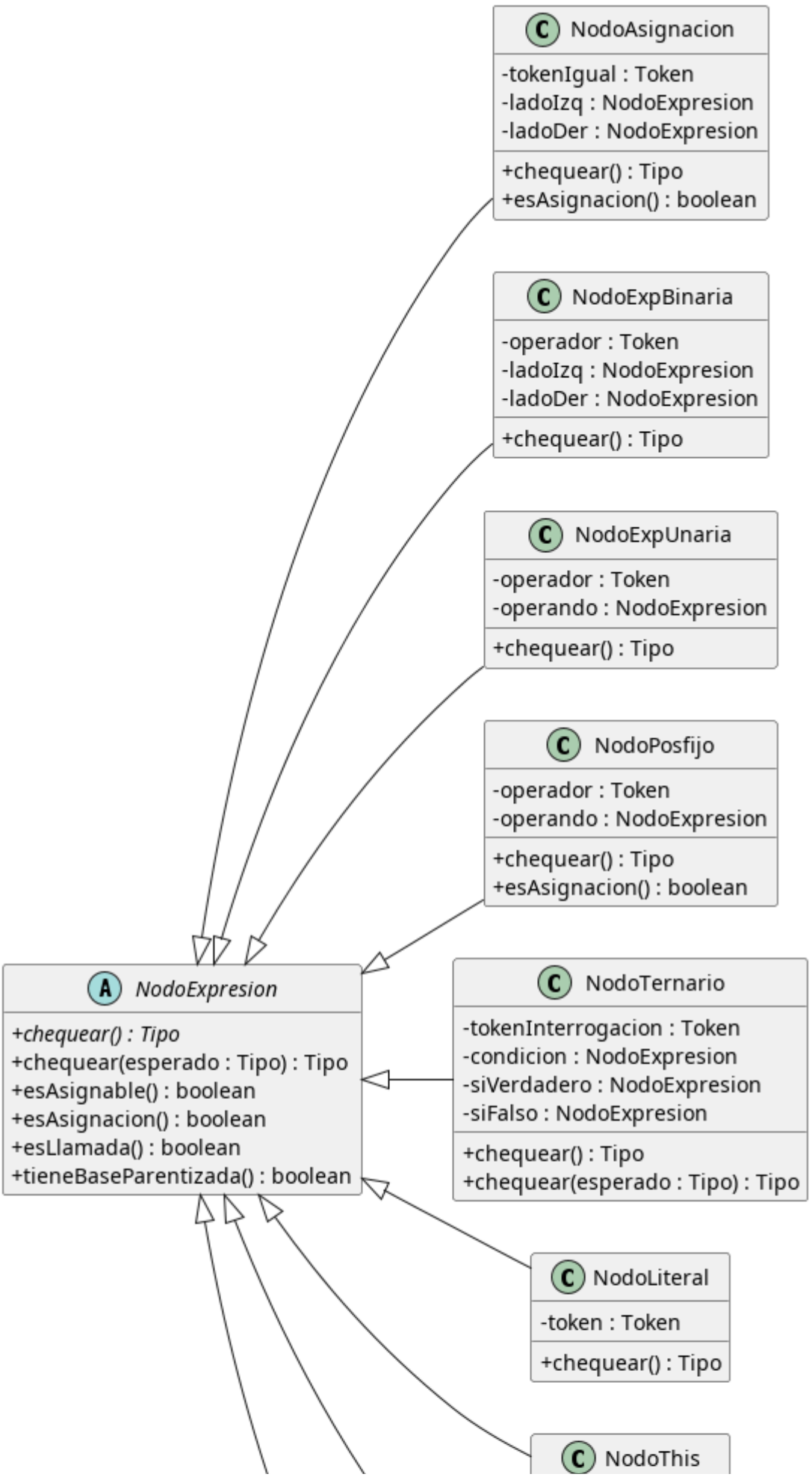


Diagrama de clases de los AST: operadores, literales y accesos



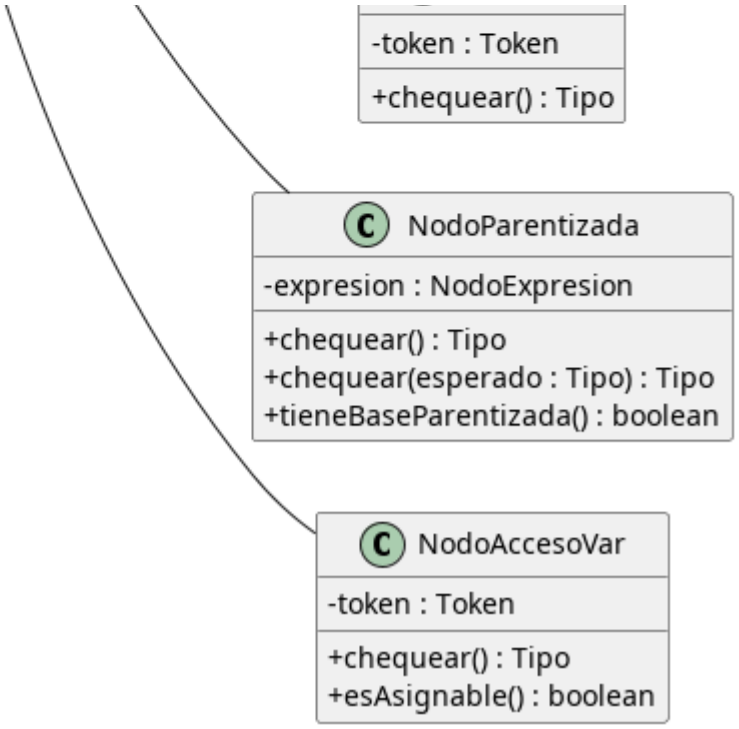
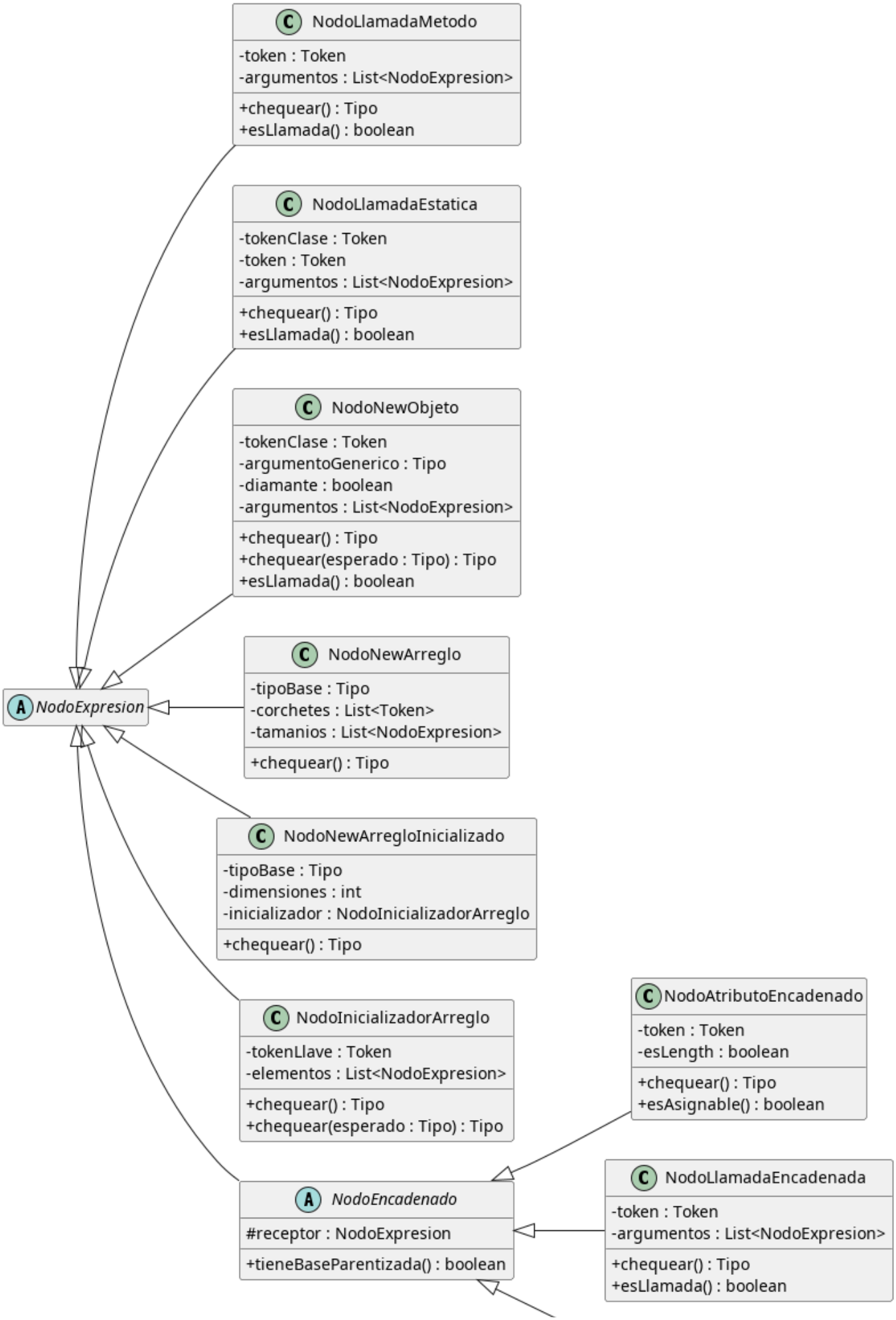
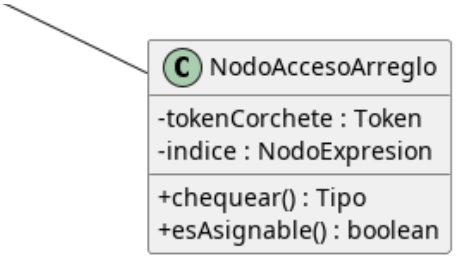

-token : Token
+chequear() : Tipo
NodoParentizada
-expresion : NodoExpresion
+chequear() : Tipo
+chequear(esperado : Tipo) : Tipo
+tieneBaseParentizada() : boolean
NodoAccesoVar
-token : Token
+chequear() : Tipo
+esAsignable() : boolean

Diagrama de clases de los AST: llamadas, creaciones y encadenados





Instrucciones de Compilación y Uso

Requisitos previos

- JDK 21 instalado
- No requiere tener Gradle instalado previamente (el proyecto incluye el Gradle Wrapper)

Compilación

Desde la raíz del proyecto, ejecutar:

Linux / macOS: `./gradlew jar`

Windows: `gradlew.bat jar`

Esto genera el ejecutable en: `build/libs/Compilador.jar`

Uso

Una vez compilado, el compilador se invoca desde la línea de comandos pasando como parámetro el archivo fuente de MiniJava. El comando es el mismo en Linux, macOS y Windows, ya que se ejecuta a través de `java`:

```
java -jar build/libs/Compilador.jar programa1.java
```

Donde `programa1.java` es el archivo fuente de MiniJava a compilar (se acepta cualquier extensión).

Etapas anteriores

La clase Main siempre hará referencia a la etapa actual. Se mantienen los Mains de etapas anteriores, con nombres descriptivos, en caso de que sea necesario recrear su ejecución (e.g.: MainEtapa1).

Logros

Etapa 4 (Usa Main)

- Entrega Anticipada E4
- Ternario E4
- Atributos Inicializados – Chequeado!

- Postfijos – Chequeado
- Genericidad Avanzada E4